develop marketing materials for customers; and every access decision follows ACME's zero-trust policy that critical resources can only be accessed from company-managed devices, with extra safeguards outside business hours.

By the end of this chapter, you'll have the patterns, structure, and working code to move from "we understand Cedar" to "we're running Cedar in production." This is the practice phase where your policies become as essential to your system's architecture as your APIs and data models.

WHY CEDAR FOR IMPLEMENTATION

Cedar stands out because it isn't limited to a single authorization model. It offers the flexibility to combine attributes, relationships, and roles within the same policy set, without forcing a choice early on.

ABAC flexibility: Cedar's `when` and `unless` clauses can reference any attributes on principals, resources, or the request context. This makes it easy to enforce time-based access, device trust checks, clearance levels, or geographic restrictions.

ReBAC expressiveness: Cedar treats relationships as first-class data. Policies can traverse the entity graph to verify relationships like "manager of an owner" or "member of a project team."

RBAC simplicity: Roles and groups can be modeled as entities and used directly in permit statements, providing the clarity of RBAC where it makes sense.

At ACME Corp, the identity team can implement RBAC-style group checks, ReBAC-style relationship checks, and ABAC-style contextual conditions across a set of Cedar policies. Cedar's evaluator considers all relevant policies for a request, allowing different patterns to work together to produce a single decision. This flexibility is powerful because most real-world authorization requirements don't fit neatly into just ABAC, just ReBAC, or just RBAC, and Cedar allows you to combine them in whatever structure makes sense for maintainability.